Linux kernel, which is a part of the board support package. The board file resides in *arch/ directory* in Linux (for example, the board file for the Beagle board is in *arch/arm/mach-omap2/board-omap3beagle.c*). The *struct spi_device* is not directly written but a different structure called *struct spi_board_info* is filled and registered, which creates the *struct spi_device* in the kernel automatically and links it to the SPI master driver that contains the routines to read and write on the SPI bus. The *struct spi_board_info* for RTC DS1347 can be written in the board file as follows:

```
struct spi_board_info spi_board_info[] __initdata = {
.modalias = "ds1347",
.bus_num = 1,
.chip_select = 1,
};
```

Modalias is the name of the driver used to identify the driver that is related to this SPI slave device—in which case the driver will have the same name. *Bus_num* is the number of the SPI bus. It is used to identify the SPI master driver that controls the bus to which this SPI slave device is connected. *Chip_select* is used in case the SPI bus has multiple chip select pins; then this number is used to identify the chip select pin to which this SPI slave device is connected.

The next step is to register the struct *spi_board_info* with the Linux kernel. In the board file initialisation code, the structure is registered as follows:

```
spi_register_board_info(spi_board_info, 1);
```

The first parameter is the array of the struct *spi_board_info* and the second parameter is the number of elements in the array. In the case of RTC DS1347, it is one. This API will check if the bus number specified in the *spi_board_info* structure matches with any of the master driver bus numbers that are registered with the Linux kernel. If any of them do match, it will create the struct *spi_device* and initialise the fields of the *spi_device* structure as follows:

```
master = spi_master driver which has the same bus number as
bus_num in the spi_board_info structure.
chip_select = chip_select of spi_board_info
modalias = modalias of spi_board_info
```

After initialising the above fields, the structure is registered with the Linux SPI subsystem. The following are the fields of the struct *spi_device*, which will be initialised by the SPI protocol driver as needed by the driver, and if not needed, will be left empty.

max_speed_hz = the maximum rate of transfer to the bus.
bits_per_word = the number of bits per transfer.
mode = the mode in which the SPI device works.

In the above specified manner, any SPI slave device is registered with the Linux kernel and the struct *spi_device* is created and linked to the Linux SPI subsystem to describe the device. This *spi_device* struct will be passed as a parameter to the SPI protocol driver probe routine when the SPI protocol driver is loaded.

Registering the RTC DS1347 SPI protocol driver

The driver is the medium through which the kernel interacts with the device connected to the system. In case of the SPI device, it is called the SPI protocol driver. The first step in writing an SPI protocol driver is to fill the *struct spi_driver* structure. For RTC DS1347, the structure is filled as follows:

```
static struct spi_driver ds1347_driver = {
     .driver = {
       .name = "ds1347",
       .owner = THIS_MODULE,
     },
     .probe = ds1347_probe,
};
```

The *name* field has the name of the driver (this should be the same as in the *modalias* field of the struct *spi_board_info*). 'Owner' is the module that owns the driver, *THIS_MODULE* is the macro that refers to the current module in which the driver is written (the 'owner' field is used for reference counting of the module owning the driver). The probe is the most important routine that is called when the device and the driver are both registered with the kernel.

The next step is to register the driver with the kernel. This is done by a macro *module_spi_driver (struct spi_driver *)*. In the case of RTC DS1347, the registration is done as follows:

```
module_spi_driver(ds1347_driver);
```

The probe routine of the driver is called if any of the following cases are satisfied:

1. If the device is already registered with the kernel and then the driver is registered with the kernel.
2. If the driver is registered first, then when the device is registered with the kernel, the probe routine is called.

In the probe routine, we need to read and write on the SPI bus, for which certain common steps need to be followed. These steps are written in a generic routine, which is called throughout to avoid duplicating steps. The generic routines are written as follows:

1. First, the address of the SPI slave device is written on the SPI bus. In the case of the RTC DS1347, the address should contain its most significant bit, reset for the write operation (as per the DS1347 datasheet).
2. Then the data is written to the SPI bus.

Since this is a common operation, a separate routine *ds1347_*